

IFTS TECNICHE PER LA PROGETTAZIONE E LO SVILUPPO DI
APPLICAZIONI INFORMATICHE

Autori Tesina Autori Martina Salvatore

programmazione IOT linguaggio Arduino C

Sensore Radar

Enti formativi POST DIPLOMA



UNIVERSITÀ
DEGLI STUDI
DI TORINO



L'Istituto Tecnico Tecnologico Statale "Carlo Grassi" di Torino 1974.

La sua fondazione rispose alle necessità di un territorio in forte espansione industriale, specialmente nel settore **metalmeccanico**. Negli anni Ottanta, l'istituto ha vissuto la sua massima crescita parallelamente allo sviluppo dell'**industria aeronautica**, diventando un punto di riferimento per l'innovazione tecnologica e la formazione tecnica a Torino.

Oggi l'istituto offre diversi percorsi di studio:

- **Trasporti e Logistica** (conduzione e costruzione del mezzo aereo).
- **Meccatronica**.
- **Elettrotecnica ed Elettronica**.
- **Informatica**.




ITTS C. Grassi

Trasporti e Logistica Conduzione del Mezzo Aereo • Costruzione del Mezzo Aereo
Elettrotecnica • Informatica • Meccatronica • Biennio



Autore Salvatore

Per prima cosa abbiamo realizzato un supporto in cartone per collegare il sensore a ultrasuoni al servomotore. Il supporto è stato piegato, incollato e fissato al servomotore utilizzando una vite.

Inoltre, abbiamo applicato un pin header sul quale sono stati saldati quattro ponticelli per il collegamento del sensore.

Abbiamo collegato il sensore a ultrasuoni **HC-SR04** ai pin 10 e 11 e il servomotore al pin 12 della scheda **Arduino**.

A questo punto dobbiamo creare un codice e caricarlo sulla scheda Arduino per consentire l'interazione tra Arduino e l'IDE di Elaborazione (Processing).

Riceveremo i valori dell'angolo e della distanza misurati dal sensore dalla scheda Arduino nell'IDE di Elaborazione utilizzando la funzione `serialEvent()`, che legge i dati dalla porta seriale. I valori dell'angolo e della distanza verranno inseriti nelle variabili `iAngle` e `iDistance`.

Queste variabili saranno poi utilizzate per disegnare il radar, le linee, gli oggetti rilevati e parte del testo all'interno dell'interfaccia grafica.

Se vuoi, posso anche aiutarti a scrivere:

- il **codice Arduino** per il sensore e il servo
- il **codice Processing** per il radar

```
// Include la libreria Servo
#include <Servo.h>
```

```

// Definisce i pin Trig ed Echo del sensore a ultrasuoni
const int trigPin = 10;
const int echoPin = 11;

// Variabili per la durata e la distanza
long durata;
int distanza;

Servo myServo; // Crea un oggetto servo per controllare il servomotore

void setup() {
  pinMode(trigPin, OUTPUT); // Imposta il TrigPin come OUTPUT
  pinMode(echoPin, INPUT); // Imposta l'EchoPin come INPUT
  Serial.begin(9600);      // Avvia la comunicazione seriale
  myServo.attach(12);      // Collega il servomotore al pin 12
}

void loop() {

  // Ruota il servomotore da 15 a 165 gradi
  for (int i = 15; i <= 165; i++) {
    myServo.write(i);
    delay(30);
    distanza = calcolaDistanza();

    Serial.print(i);
    Serial.print(",");
    Serial.print(distanza);
    Serial.print(".");
  }

  // Ruota il servomotore da 165 a 15 gradi
  for (int i = 165; i > 15; i--) {
    myServo.write(i);
    delay(30);
    distanza = calcolaDistanza();

    Serial.print(i);
    Serial.print(",");
    Serial.print(distanza);
    Serial.print(".");
  }
}

// Funzione per il calcolo della distanza
int calcolaDistanza() {

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Imposta il trigPin su HIGH per 10 microsecondi
  digitalWrite(trigPin, HIGH);

```

```
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

// Legge il tempo di ritorno dell'onda sonora
durata = pulseIn(echoPin, HIGH);

// Calcola la distanza (velocità del suono 0.034 cm/us)
distanza = durata * 0.034 / 2;

return distanza;
}
```

Componenti utilizzati

- Sensore a ultrasuoni **HC-SR04**
 - Servomotore
 - Scheda **Arduino Uno** (o compatibile)
 - IDE **Processing** per la visualizzazione grafica
-



Come funziona il sistema

1 Movimento del servomotore

Il servomotore è collegato al pin 12.

Nel `loop( )` Arduino fa ruotare il servo:

- da **15° a 165°**
- poi da **165° a 15°**

Questo movimento continuo simula la scansione di un radar.

2 Misurazione della distanza

Per ogni angolo:

1. Arduino attiva il **Trig** del sensore per 10 microsecondi.
2. Il sensore invia un'onda ultrasonica.
3. Quando l'onda colpisce un oggetto, torna indietro.
4. Il pin **Echo** misura il tempo impiegato dall'onda per tornare.
5. Arduino calcola la distanza con la formula:

$distanza = 2 \times durata \times 0.034$

- $0.034 \text{ cm}/\mu\text{s} = \text{velocità del suono}$

- Si divide per 2 perché il tempo è di andata e ritorno.
-

3 Invio dei dati al computer

Per ogni posizione del servo, Arduino invia tramite porta seriale:

angolo, distanza.

Esempio:

90, 35.

Significa:

- Angolo = 90°
- Oggetto rilevato a 35 cm

Il punto (.) serve a Processing per capire dove finisce ogni dato.

4 Visualizzazione in Processing

Nel programma Processing:

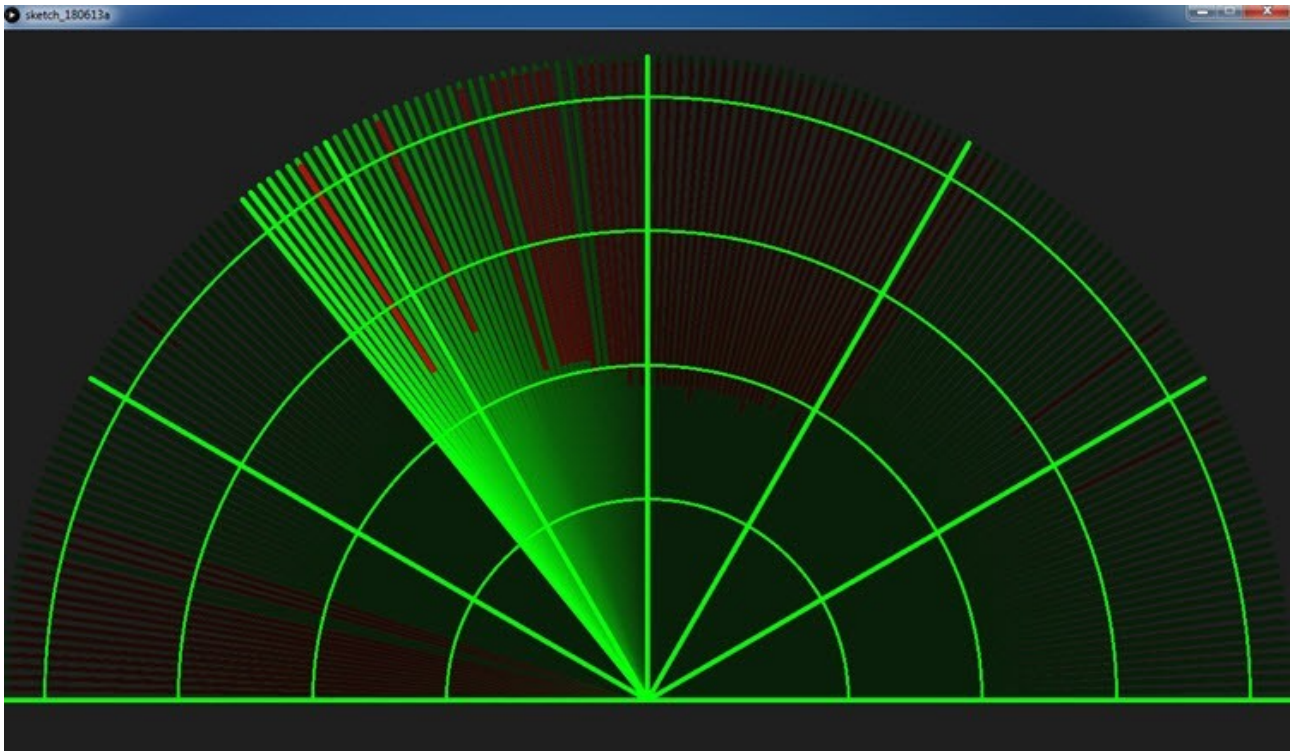
- La funzione `serialEvent()` legge i dati ricevuti.
 - I valori vengono salvati in:
 - `iAngle`
 - `iDistance`
 - Con questi valori vengono disegnati:
 - la linea del radar
 - gli oggetti rilevati
 - le distanze
 - il testo informativo
-



In sintesi

Il sistema funziona come un piccolo **radar**:

1. Il servo ruota.
2. Il sensore misura la distanza.
3. Arduino invia angolo e distanza.
4. Processing disegna tutto a schermo in tempo reale.



Ecco il codice sorgente di elaborazione completo del radar Arduino
Autore Martina

```
// Radar Arduino - Processing
// Autore rifatto da Martina Mirandi
// Versione corretta e leggibile

import processing.serial.*; // libreria per la comunicazione seriale

Serial myPort; // oggetto seriale

// Variabili per i dati ricevuti da Arduino
String data = "";
String angleStr = "";
String distanceStr = "";
String noObject;

float pixDistance;
int iAngle, iDistance;

// Font
PFont orcFont;

void setup() {
  size(1920, 1080); // cambiare in base alla risoluzione dello schermo
  smooth();

  // Inizializza la comunicazione seriale (modificare la porta COM)
  myPort = new Serial(this, "COM4", 9600);
  myPort.bufferUntil('.'); // legge i dati fino al carattere '.'
}
```

```

    orcFont = createFont("OCRAExtended-30.vlw", 30);
    textFont(orcFont);
}

void draw() {
    background(98, 245, 31); // sfondo verde

    drawRadar();
    drawLine();
    drawObject();
    drawText();
}

// Funzione che legge i dati seriali da Arduino
void serialEvent(Serial myPort) {
    data = myPort.readStringUntil('.');
    if (data != null) {
        data = data.substring(0, data.length() - 1); // rimuove il punto finale
        int commaIndex = data.indexOf(',');
        if (commaIndex > 0) {
            angleStr = data.substring(0, commaIndex);
            distanceStr = data.substring(commaIndex + 1);

            iAngle = int(angleStr);
            iDistance = int(distanceStr);
        }
    }
}

// Disegna il radar
void drawRadar() {
    pushMatrix();
    translate(width / 2, height - height * 0.074f); // centro radar

    noFill();
    strokeWeight(2);
    stroke(98, 245, 31);

    // Disegna gli archi concentrici
    arc(0, 0, width * 0.938f, width * 0.938f, PI, TWO_PI);
    arc(0, 0, width * 0.73f, width * 0.73f, PI, TWO_PI);
    arc(0, 0, width * 0.521f, width * 0.521f, PI, TWO_PI);
    arc(0, 0, width * 0.313f, width * 0.313f, PI, TWO_PI);

    // Disegna le linee degli angoli
    line(-width / 2, 0, width / 2, 0);
    for (int a = 30; a <= 150; a += 30) {
        float rad = radians(a);
        line(0, 0, -width / 2 * cos(rad), -width / 2 * sin(rad));
    }
}

```

```

    popMatrix();
}

// Disegna gli oggetti rilevati dal sensore
void drawObject() {
    if (iDistance > 0 && iDistance < 40) {
        pixDistance = iDistance * (height * 0.025f); // scala da cm a pixel

        pushMatrix();
        translate(width / 2, height - height * 0.074f);
        stroke(255, 10, 10);
        strokeWeight(9);

        float rad = radians(iAngle);
        line(pixDistance * cos(rad), -pixDistance * sin(rad),
            (width / 2) * cos(rad), -(width / 2) * sin(rad));

        popMatrix();
    }
}

// Disegna la linea del radar
void drawLine() {
    pushMatrix();
    translate(width / 2, height - height * 0.074f);
    stroke(30, 250, 60);
    strokeWeight(9);

    float rad = radians(iAngle);
    line(0, 0, height * 0.12f * cos(rad), -height * 0.12f * sin(rad));

    popMatrix();
}

// Disegna il testo sullo schermo
void drawText() {
    pushMatrix();
    noStroke();

    if (iDistance > 40) {
        noObject = "Fuori intervallo";
    } else {
        noObject = "Nell'intervallo";
    }

    fill(0);
    rect(0, height - height * 0.0648f, width, height * 0.0648f);
    fill(98, 245, 31);

    textSize(25);
    text("10 cm", width * 0.615f, height - height * 0.0833f);
    text("20 cm", width * 0.719f, height - height * 0.0833f);
}

```

```

text("30 cm", width * 0.823f, height - height * 0.0833f);
text("40 cm", width * 0.927f, height - height * 0.0833f);

textSize(40);
text("Oggetto: " + noObject, width * 0.125f, height - height * 0.0277f);
text("Angolo: " + iAngle + "°", width * 0.52f, height - height * 0.0277f);
text("Distanza: ", width * 0.74f, height - height * 0.0277f);
if (iDistance < 40) {
  text(iDistance + " cm", width * 0.775f, height - height * 0.0277f);
}

popMatrix();
}

```

Parte Arduino: sensore a ultrasuoni + servomotore

Componenti principali:

- **HC-SR04** → sensore a ultrasuoni che misura la distanza degli oggetti.
- **Servomotore** → ruota il sensore a ultrasuoni da 15° a 165°.
- **Arduino** → controlla servo e sensore e invia dati al PC via seriale.

Funzionamento:

1. Movimento del servomotore

- Arduino ruota il servomotore da 15° a 165° e poi torna indietro.
- Ogni passo del servo è di 1° con un delay di 30 ms, così il movimento è fluido.
- Questo movimento simula la scansione di un radar.

2. Misurazione della distanza

- Arduino invia un impulso di 10 microsecondi al pin **Trig** del sensore.
- Il sensore emette un'onda ultrasonica che rimbalza sugli oggetti vicini.
- Il pin **Echo** legge quanto tempo impiega l'onda a tornare.
- La distanza viene calcolata con la formula:

$$(cm) (\mu s) \text{distanza} (cm) = 2 \text{durata} (\mu s) \times 0.034$$
- Si divide per 2 perché il tempo misurato è andata e ritorno.

3. Invio dati al computer

- Arduino invia ogni rilevamento via **porta seriale** nel formato:
 angolo,distanza.
 Esempio: 90,35. → angolo = 90°, distanza = 35 cm.
- Il punto finale (.) indica a Processing dove finisce un dato.

2 Parte Processing: radar grafico sul computer

Componenti principali:

- **Processing IDE** → riceve i dati da Arduino e disegna il radar.
- **SerialEvent()** → funzione che legge i dati dalla porta seriale.

Funzionamento:

1. Ricezione dati

- La funzione `serialEvent()` legge ogni pacchetto di dati fino al carattere ..
- Divide il pacchetto in due valori:
 - **iAngle** → angolo del servo.
 - **iDistance** → distanza dell'oggetto rilevato.
- Converte le stringhe in numeri interi per usarli nel disegno.

2. Disegno del radar

- `drawRadar()` → disegna:
 - archi concentrici per rappresentare le distanze (10 cm, 20 cm, ecc.)
 - linee di angolo (30°, 60°, 90°, 120°, 150°) che aiutano a capire la direzione.
- Tutto viene centrato in basso a metà dello schermo per simulare un radar reale.

3. Disegno della linea di scansione

- `drawLine()` → disegna la linea verde che ruota insieme al servo di Arduino, indicando la direzione di rilevamento.

4. Disegno degli oggetti rilevati

- `drawObject()` → disegna una linea rossa dal radar fino all'oggetto rilevato.
- La distanza del sensore (cm) viene trasformata in pixel per lo schermo.

5. Testo informativo

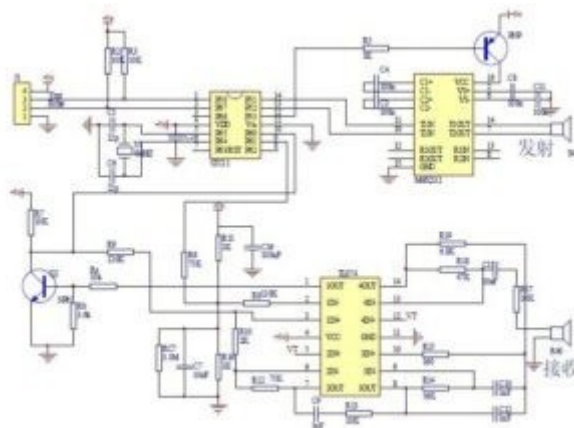
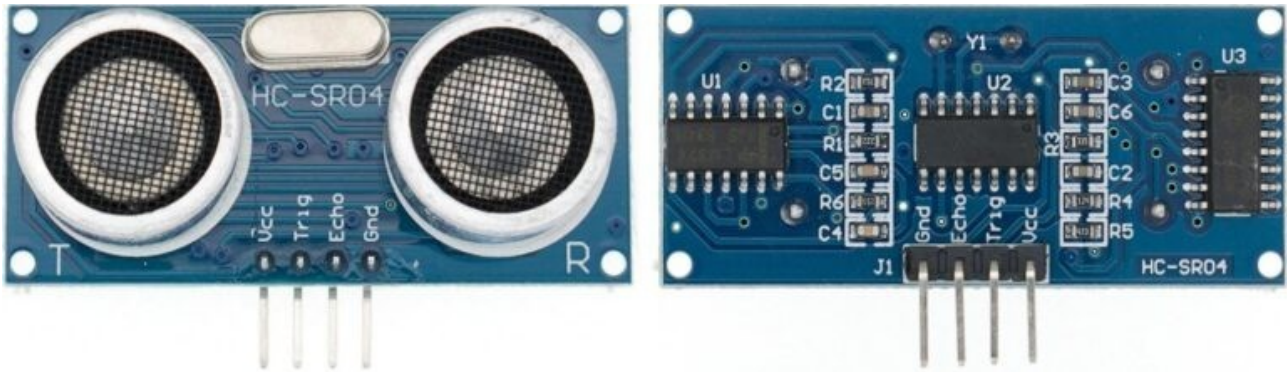
- `drawText()` → mostra sullo schermo:
 - stato oggetto: "Nell'intervallo" o "Fuori intervallo"
 - angolo rilevato
 - distanza rilevata in cm
 - riferimenti di scala (10 cm, 20 cm, ecc.)
 - Testi orientati in modo chiaro per ogni arco e angolo.
-

◆ **In sintesi**

Arduino: controlla servo e sensore → calcola distanza → invia dati al PC.

Processing: legge i dati → li trasforma in grafica → mostra radar in tempo reale con linea di scansione e oggetti rilevati.

Il risultato è un radar virtuale che simula la scansione di un vero radar, mostrando gli oggetti rilevati in tempo reale con grafica dinamica.



Il sensore HC-SR04 non ci da direttamente la misura della distanza dell'oggetto, ma lo si utilizza per emettere un treno di impulsi ad ultrasuoni (ne saranno emessi 8 quando verrà portato per 10 microsecondi a stato alto il suo pin Trigger). Gli impulsi sonori saranno propagati nell'ambiente circostante e, se incontrano uno ostacolo, torneranno indietro verso il sensore che li ha emessi. Quando il sensore “sentirà” il ritorno dell'impulso sonoro porterà a stato basso il suo pin Echo (che nel frattempo era stato portato automaticamente alto). Misurando il tempo che intercorre tra l'emissione del segnale sonoro ed il suo ritorno potremo calcolare la distanza dell'ostacolo sul quale è rimbalzato. Come?

La velocità del suono è la velocità con cui un suono si propaga in un certo ambiente, detto mezzo. La velocità del suono varia a seconda del mezzo (ad esempio, il suono si propaga più velocemente nell'acqua che non nell'aria), e anche al variare delle proprietà del mezzo, specialmente con la sua temperatura.

Il nostro mezzo è l'aria, nella quale la velocità del suono è di 331,2 metri al secondo (1 192,32 km/h) a 0 °C e di 343,1 m/s (1 235,16 km/h) a 20 °C temperatura nella quale normalmente

utilizziamo i sensori agli ultrasuoni (per chi vuole approfondire l'argomento della velocità del suono si rimanda alla pagina di Wikipedia).

Detto questo convertiamo la velocità del suono da 343,1 m/s a 0,03431 cm/microsecondi.

Dato che lo spazio percorso = velocità x il tempo, la nuova formula sarà:

Spazio (cm) = 0,03431 x t (microsecondi)

Dobbiamo però ricordarci che la distanza percorsa dal suono è doppia rispetto alla distanza dell'ostacolo, in quanto l'impulso sonoro deve andare verso l'ostacolo e tornare indietro, quindi anche il tempo impiegato in totale è doppio.

La nuova formula sarà quindi:

Spazio = 0.03431 x t / 2

che equivale a:

Spazio = 0,017155 x t

In alcuni esempi sulla rete si potrà trovare la formula scritta anche così:

Spazio = t / 58.3 (deriva da aver diviso 1 su 0,017155) e spesso viene approssimato a:

Spazio = t / 58 per sapere la distanza in cm, oppure:

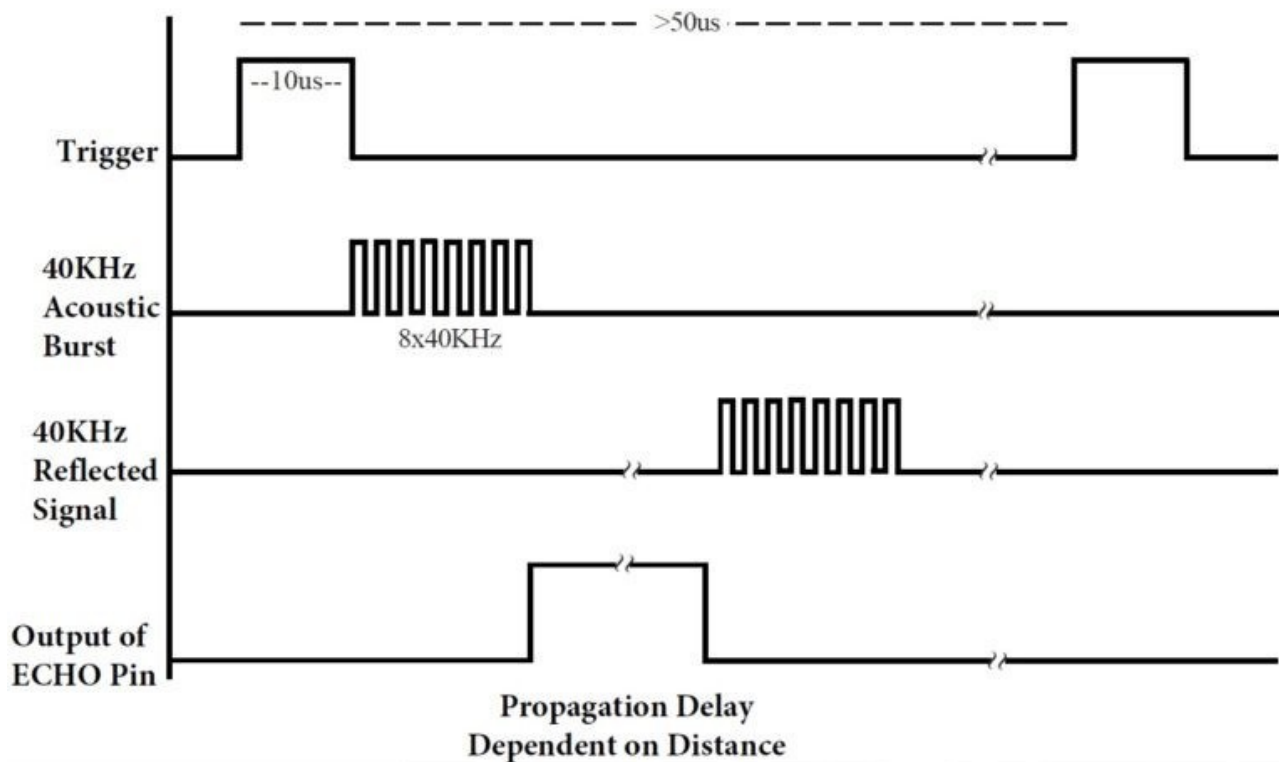
Spazio = t / 148 per sapere la distanza in pollici.

Messa da parte la matematica vediamo come utilizzare l'HC SR04 con Arduino

Codice sorgente primo sketch che stato sviluppato

```
#define trigPin 6
#define echoPin 7
long durata, cm;
void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}
void loop()
{
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  durata = pulseIn(echoPin, HIGH);
  cm = durata / 58; // per i pollici la formula è durata / 148;
  Serial.print("Cm = ");
  Serial.println(cm);
  Serial.println();
}
```

Segnale



Durante il tirocinio presso la cooperativa Arcobaleno, ho avuto l'opportunità di approfondire la gestione dei Big Data. In particolare, mi sono occupato dell'automazione e della manipolazione di dati in formato JSON.

La manipolazione dei dati JSON consiste nel **leggere, modificare, filtrare e organizzare** le informazioni contenute nei file o negli oggetti JSON, in modo che possano essere facilmente analizzate o utilizzate da software e applicazioni. Ad esempio, si possono estrarre solo alcune informazioni specifiche, combinare più dataset o aggiornare valori automaticamente senza intervento manuale.

1. Struttura generale

- **HTML (<html>, <head>, <body>):** definisce il contenuto e la struttura della pagina.
- **CSS (<style>):** definisce lo stile, colori, margini, dimensioni e layout.
- **JavaScript (<script>):** gestisce la logica dell'app, cioè l'inserimento, la memorizzazione e la visualizzazione dei dati.

2. Interfaccia utente

La pagina contiene:

1. **Un titolo:** "Tecniche di automatizzazione e manipolazione dati strutturati".
 2. **Un modulo (<form>)** con campi per inserire informazioni:
 - `cod_id`: codice identificativo
 - nome e cognome
 - `eta` (numero)
 - `mansione`
 - `tipo_vegetale` e `tipo_materiale`
 3. **Pulsante “Invia Dati”:** chiama la funzione `dati()` di JavaScript.
 4. **Sezioni per visualizzare i dati:**
 - `<pre id="output">`: mostra i dati in formato JSON leggibile.
 - `<div id="tabella">`: mostra i dati in una tabella.
 5. **Pulsante “Elimina Record Selezionato”:** chiama la funzione `eliminaRecord()`.
-

3. Stile CSS

- Sfondo **lime** e font Arial.
 - Il contenitore principale ha larghezza massima di **600px** e margini centrati.
 - Tabelle con bordi e padding.
 - Pulsanti con colore verde e effetto hover.
 - Layout flessibile con **Flexbox**.
-

4. Logica JavaScript

a) Archiviazione dei dati

- I dati inseriti vengono salvati in un **array** `arr` e memorizzati nel **localStorage** del browser.
- Questo permette di **mantenere i dati anche dopo aver chiuso il browser**.

```
localStorage.setItem("arr", JSON.stringify(arr));  
arr = JSON.parse(localStorage.getItem("arr"));
```

b) Inserimento dati (`dati()`)

1. Prende i valori dai campi del modulo.

2. Crea un oggetto con questi dati più la **data corrente**.
3. Aggiunge l'oggetto all'array `arr`.
4. Salva l'array in `localStorage`.
5. Mostra i dati in JSON (output) e aggiorna la tabella (`aggiornaTabella()`).

```
var biologia_VEGETALI = {  
  cod_id, nome, cognome, eta, mansione, tipo_vegetale, tipo_materiale,  
  data: data.getFullYear() + "/" + (data.getMonth() + 1) + "/" +  
  data.getDate()  
};  
arr.push(biologia_VEGETALI);
```

c) Visualizzazione dati (`aggiornaTabella()`)

- Costruisce una tabella HTML con tutti i dati presenti nell'array.
- Ogni riga ha:
 - Un radio button per selezionare il record
 - Tutti i campi del record

```
for (var i = 0; i < arr.length; i++) {  
  html += "<tr>";  
  html += "<td><input type='radio' name='record' value='" + i + "'></td>";  
  html += "<td>" + arr[i].cod_id + "</td>";  
  ...  
  html += "</tr>";  
}
```

d) Eliminazione record (`eliminaRecord()`)

1. Trova quale radio button è selezionato.
2. Rimuove l'elemento corrispondente dall'array `arr`.
3. Aggiorna `localStorage`.
4. Aggiorna la tabella in pagina.

```
arr.splice(selectedIndex, 1);  
localStorage.setItem("arr", JSON.stringify(arr));  
aggiornaTabella();  
Se nessun record è selezionato, mostra un alert.
```

e) Caricamento dati iniziale (`caricaDati()`)

All'avvio della pagina, recupera eventuali dati salvati nel `localStorage` e li mostra subito nella tabella.

```
if (localStorage.getItem("arr")) {  
  arr = JSON.parse(localStorage.getItem("arr"));  
  count = arr.length;  
  aggiornaTabella();  
}
```

}

5. Cosa fa complessivamente il software

1. Permette di **inserire dati** su persone e vegetali.
2. Visualizza i dati in **JSON** e in **tabella**.
3. Permette di **selezionare e cancellare record**.
4. Salva tutto nel **localStorage**, quindi i dati non si perdono chiudendo il browser.
5. È completamente **client-side**, non serve serve

Autore del codice Brezza Salvatore

```
<!DOCTYPE html>
<html lang="it">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Inserisci Dati Vegetali</title>
<style>
body {
font-family: Arial, sans-serif;
background-color: lime; /* Colore di sfondo lime */
margin: 0;
padding: 0;
display: flex;
justify-content: center;
align-items: flex-start;
flex-direction: column;
height: 100vh;
color: #333;
padding: 20px;
}

h2 {
text-align: center;
margin-top: 30px;
margin-bottom: 20px;
}
```

```
/* Ridurre la larghezza del contenitore principale */  
div {  
width: 100%;  
max-width: 600px; /* Imposta un massimo di 600px per il contenitore */  
margin: 0 auto;  
}
```

```
label {  
display: block;  
margin: 10px 0 5px;  
}
```

```
button {  
background-color: #4CAF50;  
color: white;  
padding: 8px;  
border: none;  
border-radius: 4px;  
font-size: 1em;  
cursor: pointer;  
width: 100%;  
}
```

```
button:hover {  
background-color: #45a049;  
}
```

```
h3 {  
margin-top: 20px;  
text-align: center;  
}
```

```
table {  
width: 100%;  
margin-top: 20px;  
border-collapse: collapse;  
}
```

```
table, th, td {  
border: 1px solid #ddd;  
}
```

```
th, td {  
padding: 8px;  
text-align: left;  
}
```

```
.actions button {  
width: auto;  
padding: 10px 20px;  
}
```

```
}  
</style>
```

```
</head>  
<body>
```

```
<div>  
<h2>Tecniche di automatizzazione e manipolazione dati strutturati</h2>
```

```
<form id="formDati">  
<label for="cod_id">Codice ID:</label>  
<input type="text" id="cod_id" name="cod_id">
```

```
<label for="nome">Nome:</label>  
<input type="text" id="nome" name="nome">
```

```
<label for="cognome">Cognome:</label>  
<input type="text" id="cognome" name="cognome">
```

```
<label for="eta">Età:</label>  
<input type="number" id="eta" name="eta">
```

```
<label for="mansione">Mansione:</label>  
<input type="text" id="mansione" name="mansione">
```

```
<label for="tipo_vegetale">Tipo Vegetale:</label>  
<input type="text" id="tipo_vegetale" name="tipo_vegetale">
```

```
<label for="tipo_materiale">Tipo Materiale:</label>  
<input type="text" id="tipo_materiale" name="tipo_materiale">
```

```
<button type="button" onclick="dati()">Invia Dati</button>
</form>
```

```
<h3>Dati Inseriti:</h3>
<pre id="output"></pre>
```

```
<h3>Tabella Dati:</h3>
<div id="tabella"></div>
```

```
<div class="actions">
<button type="button" onclick="eliminaRecord()">Elimina Record Selezionato</button>
</div>
</div>
```

```
<script>
var arr = []; // Array per memorizzare gli oggetti
var count = 0;
```

```
// Carica i dati da localStorage se ci sono
function caricaDati() {
if (localStorage.getItem("arr")) {
arr = JSON.parse(localStorage.getItem("arr"));
count = arr.length;
aggiornaTabella();
}
}
```

```
// Funzione che viene chiamata quando si clicca sul pulsante per inserire i dati
function dati() {
var data = new Date(); // Crea un nuovo oggetto Data
```

```
// Ottieni i valori dai campi di input
var cod_id = document.getElementById("cod_id").value;
var nome = document.getElementById("nome").value;
var cognome = document.getElementById("cognome").value;
var eta = document.getElementById("eta").value;
var mansione = document.getElementById("mansione").value;
var tipo_vegetale = document.getElementById("tipo_vegetale").value;
var tipo_materiale = document.getElementById("tipo_materiale").value;
```

```
// Crea l'oggetto con i dati inseriti
var biologia_VEGETALI = {
```

```
cod_id: cod_id,  
nome: nome,  
cognome: cognome,  
eta: eta,  
mansione: mansione,  
tipo_vegetale: tipo_vegetale,  
tipo_materiale: tipo_materiale,  
data: data.getFullYear() + "/" + (data.getMonth() + 1) + "/" + data.getDate() // Aggiungiamo la data  
direttamente qui  
};
```

```
// Aggiungi l'oggetto all'array  
arr.push(biologia_VEGETALI); // Usa push per aggiungere un nuovo elemento  
count++;
```

```
// Salva l'array in localStorage  
localStorage.setItem("arr", JSON.stringify(arr));
```

```
// Mostra i dati inseriti nella pagina  
document.getElementById("output").textContent = JSON.stringify(arr, null, 2);
```

```
// Stampa anche in console per debug  
console.log(JSON.stringify(arr));
```

```
// Aggiorna la tabella  
aggiornaTabella();  
}
```

```
// Funzione per aggiornare la tabella  
function aggiornaTabella() {  
var html = "<table>";  
html += "<tr><th>Seleziona</th><th>Codice  
ID</th><th>Nome</th><th>Cognome</th><th>Età</th><th>Mansione</th><th>Tipo  
Vegetale</th><th>Tipo Materiale</th><th>Data</th></tr>";
```

```
// Crea le righe della tabella per ogni elemento nell'array  
for (var i = 0; i < arr.length; i++) {  
html += "<tr>";  
html += "<td><input type='radio' name='record' value='" + i + "'></td>";  
html += "<td>" + arr[i].cod_id + "</td>";  
html += "<td>" + arr[i].nome + "</td>";  
html += "<td>" + arr[i].cognome + "</td>";  
html += "<td>" + arr[i].eta + "</td>";  
html += "<td>" + arr[i].mansione + "</td>";
```

```

html += "<td>" + arr[i].tipo_vegetale + "</td>";
html += "<td>" + arr[i].tipo_materiale + "</td>";
html += "<td>" + arr[i].data + "</td>";
html += "</tr>";
}

html += "</table>";
document.getElementById("tabella").innerHTML = html;
}

// Funzione per eliminare il record selezionato
function eliminaRecord() {
var selectedIndex = -1;
var radios = document.getElementsByName('record');

// Trova quale radio button è selezionato
for (var i = 0; i < radios.length; i++) {
if (radios[i].checked) {
selectedIndex = i;
break;
}
}

// Se un record è stato selezionato
if (selectedIndex >= 0) {
// Rimuovi il record dall'array
arr.splice(selectedIndex, 1);

// Salva di nuovo l'array aggiornato in localStorage
localStorage.setItem("arr", JSON.stringify(arr));

// Aggiorna la tabella
aggiornaTabella();

} else {
alert("Seleziona un record da eliminare.");
}
}

// Carica i dati all'avvio
caricaDati();
</script>

```

```
</body>
</html>
```

In questo percorso di studi abbiamo anche fatto la parte dei video giochi in javascript css html in 2D tramite itis grassi

Struttura HTML

```
<h1>ITIS GRASSI GIOCO BOLLE JAVASCRIPT</h1>
<h2>Lavoro di gruppo...</h2>
<canvas id="canvas1" width="800" height="500"></canvas>
<div class="controls">
  <!-- Pulsanti frecce per muovere il personaggio -->
</div>
```

- `<canvas>`: area grafica dove il gioco viene disegnato.
 - `.controls` con `<button>`: pulsanti touch per muovere il personaggio (su, giù, sinistra, destra).
 - `ontouchstart` e `ontouchend`: servono a gestire il tocco sui dispositivi mobili.
-

2 Stile CSS

```
body {
  font-family: Arial;
  background-color: #f0f0f0;
}
canvas { background: #ccc; }
button { width: 70px; height: 70px; ... }
@media (max-width: 768px) { ... }
```

- Imposta font, colori di sfondo e dimensioni dei pulsanti.
 - Media query: riduce dimensioni del canvas e dei pulsanti su schermi piccoli (mobile).
-

3 Script JavaScript

a) Variabili principali

```
const canvas = document.getElementById("canvas1");
const ctx = canvas.getContext("2d"); // contesto per disegnare
const sprite = new Image();
sprite.src = "d.png"; // sprite del personaggio
let x = 100, y = 200; // posizione iniziale del personaggio
let frameX = 0, frameY = 0; // frame di animazione del personaggio
const frameW = 64, frameH = 64; // dimensioni frame sprite
let direction = null; // direzione di movimento
let isMoving = false; // se il personaggio si sta muovendo
```

- `sprite` è l'immagine del personaggio.

- frameX e frameY servono per l'animazione del personaggio usando uno sprite sheet (più frame in un'unica immagine).
-

b) Bolla casuale

```
let bubble = {  
  x: Math.random() * (canvas.width - 20),  
  y: Math.random() * (canvas.height - 20),  
  radius: 10  
};
```

- La bolla appare in una posizione casuale sul canvas.
-

c) Funzioni di movimento

```
function startMove(dir) { direction = dir; isMoving = true; }  
function stopMove() { isMoving = false; }
```

- Gestiscono quando il giocatore tocca o rilascia un pulsante per muovere il personaggio.
-

d) Disegno e collisione

```
function drawBubble() {  
  ctx.beginPath();  
  ctx.fillStyle = "blue";  
  ctx.arc(bubble.x, bubble.y, bubble.radius, 0, Math.PI * 2);  
  ctx.fill();  
}
```

- Disegna la bolla come un cerchio blu.

```
function checkCollision() {  
  let dx = x + frameW/2 - bubble.x;  
  let dy = y + frameH/2 - bubble.y;  
  let distance = Math.sqrt(dx*dx + dy*dy);  
  if (distance < bubble.radius + frameW/2) {  
    document.body.style.background = getRandomColor();  
    bubble.x = Math.random() * (canvas.width - 20);  
    bubble.y = Math.random() * (canvas.height - 20);  
  }  
}
```

- Controlla se il personaggio tocca la bolla.
 - Se sì, cambia il colore dello sfondo e sposta la bolla in un'altra posizione casuale.
-

e) Colori casuali

```
function getRandomColor() {  
  return "hsl(" + Math.floor(Math.random() * 360) + ", 70%, 80%)";  
}
```

- Genera un colore casuale usando HSL (tonalità, saturazione, luminosità).
-

f) Loop principale (update)

```
function update() {
  ctx.clearRect(0, 0, canvas.width, canvas.height); // pulisce il canvas
  if (isMoving) {
    frameX = (frameX + 1) % 4; // cambia frame animazione
    if (direction === 'right' && x + frameW < canvas.width) x += 2; frameY = 2;
    if (direction === 'left' && x > 0) x -= 2; frameY = 1;
    if (direction === 'up' && y > 0) y -= 2; frameY = 3;
    if (direction === 'down' && y + frameH < canvas.height) y += 2; frameY = 0;
  }
  drawBubble();
  checkCollision();
  ctx.drawImage(sprite, frameX*frameW, frameY*frameH, frameW, frameH, x, y,
frameW, frameH);
  requestAnimationFrame(update); // richiama update ad ogni frame (~60fps)
}
```

- Pulisce lo schermo ad ogni frame.
 - Muove il personaggio e aggiorna la sua animazione.
 - Disegna la bolla e verifica collisione.
 - Ridisegna il personaggio usando il frame corretto dello sprite.
 - Richiama sé stesso con `requestAnimationFrame` per creare il loop del gioco.
-

✓ Riassunto

1. C'è un personaggio controllabile con le frecce touch.
2. C'è una bolla da prendere.
3. Quando il personaggio tocca la bolla:
 - Lo sfondo cambia colore.
 - La bolla appare in un'altra posizione casuale.
4. Il movimento usa sprite sheet per animazione fluida.
5. Tutto funziona in un loop continuo con `requestAnimationFrame`



codice

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
font-family: Arial, sans-serif;
margin: 0;
padding: 0;
background-color: #f0f0f0;
}
h1, h2 {
text-align: center;
margin: 10px;
}
canvas {
background: #ccc;
display: block;
margin: 20px auto;
}
.controls {
text-align: center;
margin-top: 20px;
}
button {
```

```

width: 70px;
height: 70px;
font-size: 24px;
margin: 5px;
background-color: #4CAF50;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
}
button:active {
background-color: #45a049;
}
@media (max-width: 768px) {
h1, h2 {
font-size: 1.2em;
}
canvas {
width: 90%;
height: auto;
}
button {
width: 60px;
height: 60px;
font-size: 18px;
}
}
</style>
</head>
<body>

```

```

<h1>ITIS GRASSI GIOCO BOLLE JAVASCRIPT</h1>

```

```

<h2>Lavoro di gruppo: abbiamo progettato un gioco delle bolle che prende le bolle e cambia i colori!</h2>

```

```

<canvas id="canvas1" width="800" height="500"></canvas>

```

```

<div class="controls">

```

```

<div>

```

```

<button ontouchstart="startMove('up')" ontouchend="stopMove()">#8593;</button>

```

```

</div>

```

```

<div>

```

```

<button ontouchstart="startMove('left')" ontouchend="stopMove()">#8592;</button>

```

```

<button ontouchstart="startMove('down')" ontouchend="stopMove()">#8595;</button>

```

```

<button ontouchstart="startMove('right')" ontouchend="stopMove()">#8594;</button>

```

</div>

</div>

<script>

```
const canvas = document.getElementById("canvas1");
const ctx = canvas.getContext("2d");
const sprite = new Image();
sprite.src = "d.png"; // Assicurati che sia nella stessa cartella
```

```
let x = 100, y = 200;
let frameX = 0, frameY = 0;
const frameW = 64, frameH = 64;
let direction = null;
let isMoving = false;
```

// Bolla

```
let bubble = {
  x: Math.random() * (canvas.width - 20),
  y: Math.random() * (canvas.height - 20),
  radius: 10
};
```

```
sprite.onload = () => requestAnimationFrame(update);
```

```
function startMove(dir) {
  direction = dir;
  isMoving = true;
}
```

```
function stopMove() {
  isMoving = false;
}
```

```
function drawBubble() {
  ctx.beginPath();
  ctx.fillStyle = "blue";
  ctx.arc(bubble.x, bubble.y, bubble.radius, 0, Math.PI * 2);
  ctx.fill();
  ctx.closePath();
}
```

```
function checkCollision() {
  let dx = x + frameW / 2 - bubble.x;
```

```

let dy = y + frameH / 2 - bubble.y;
let distance = Math.sqrt(dx * dx + dy * dy);
if (distance < bubble.radius + frameW / 2) {
// Presa!
document.body.style.background = getRandomColor();
bubble.x = Math.random() * (canvas.width - 20);
bubble.y = Math.random() * (canvas.height - 20);
}
}

function getRandomColor() {
return "hsl(" + Math.floor(Math.random() * 360) + ", 70%, 80%)";
}

function update() {
ctx.clearRect(0, 0, canvas.width, canvas.height);

if (isMoving) {
frameX = (frameX + 1) % 4;

if (direction === 'right' && x + frameW < canvas.width) {
x += 2; frameY = 2;
}
if (direction === 'left' && x > 0) {
x -= 2; frameY = 1;
}
if (direction === 'up' && y > 0) {
y -= 2; frameY = 3;
}
if (direction === 'down' && y + frameH < canvas.height) {
y += 2; frameY = 0;
}
}

drawBubble();
checkCollision();

ctx.drawImage(sprite, frameX * frameW, frameY * frameH, frameW, frameH, x, y, frameW,
frameH);
requestAnimationFrame(update);
}
</script>

</body>

```

</html>

Gioco del cannone

Mini Progetto Didattico – IFTS ITIS Grassi

Questo semplice gioco 2D è stato realizzato durante un laboratorio pratico presso l'ITIS Grassi, all'interno del percorso IFTS in Informatica e Telecomunicazioni.

Il gioco ha come obiettivo colpire le bolle blu con dei proiettili sparati da un cannone ruotabile. È stato programmato in JavaScript usando il tag `<canvas>` per la grafica.

Per calcolare la traiettoria del proiettile è stata utilizzata una proporzione trigonometrica:

$$x : y = \cos(\theta) : \sin(\theta)$$

Questa proporzione ci ha permesso di definire la direzione e la velocità dei colpi in base all'angolo del cannone. Il progetto ha aiutato la classe a comprendere meglio la relazione tra matematica, fisica e programmazione.



Spiegazione

Struttura HTML

```
<h3>Cannone 2D</h3>
<canvas id="canvas" width="600" height="400"></canvas>
<div>
  <button onclick="angle -= 5">← SX</button>
  <button onclick="angle += 5">→ DX</button>
  <button onclick="spara()">💣 Spara</button>
</div>
<div id="description"> ... </div>
```

- `<canvas>`: area dove vengono disegnati cannone, proiettili e bolle.
 - Pulsanti:
 - ← SX e → DX modificano l'angolo del cannone (`angle`).
 - 💣 Spara aggiunge un nuovo proiettile nell'array `bullets`.
 - `<div id="description">`: descrizione del progetto, scopo didattico e spiegazione matematica (trigonometria).
-

2 CSS

```
body { text-align:center; background:#eef; font-family:sans-serif; }
canvas { background:#ccc; border:1px solid #000; }
button { margin:5px; padding:10px 15px; font-size:16px; }
```

- Centra tutto e imposta colori di sfondo.
 - Canvas con bordo nero.
 - Pulsanti leggibili e spaziati.
-

3 Variabili JavaScript principali

```
let angle = 0; // angolo del cannone in gradi
const cannon = { x:300, y:350, len:60 }; // posizione e lunghezza cannone
const bullets = []; // array dei proiettili
let bolle = []; // array delle bolle
```

- `angle`: angolo corrente del cannone.
 - `cannon`: coordinate della base del cannone e lunghezza della canna.
 - `bullets`: contiene tutti i proiettili sparati.
 - `bolle`: contiene le bolle da colpire.
-

4 Funzione creaBolle()

```
function creaBolle() {
  bolle = [];
  for (let i = 0; i < 10; i++) {
    bolle.push({
      x: Math.random()*560+20,
      y: Math.random()*200+20,
      r: 15,
      colpita: false
    });
  }
}
```

- Crea 10 bolle in posizioni casuali nella parte superiore del canvas.
 - `colpita: false` indica che la bolla non è stata ancora colpita.
-

5 Funzione spara()

```
function spara() {
  const rad = angle * Math.PI / 180; // converte angolo in radianti
  bullets.push({
    x: cannon.x + cannon.len * Math.cos(rad),
    y: cannon.y + cannon.len * Math.sin(rad),
    vx: 5 * Math.cos(rad),
    vy: 5 * Math.sin(rad)
  });
}
```

- Calcola posizione iniziale del proiettile alla punta del cannone.
 - `vx` e `vy` definiscono la velocità lungo X e Y usando **trigonometria**:
 - $\cos(\theta)$ per la componente X
 - $\sin(\theta)$ per la componente Y
 - Permette di sparare in qualsiasi direzione a seconda dell'angolo.
-

6 Funzione disegna()

Questa funzione è il **loop principale del gioco**.

a) Pulizia canvas

```
ctx.clearRect(0, 0, 600, 400);
```

b) Disegna il cannone

```
ctx.strokeStyle = "black";
ctx.lineWidth = 8;
const rad = angle * Math.PI / 180;
ctx.beginPath();
ctx.moveTo(cannon.x, cannon.y);
ctx.lineTo(cannon.x + cannon.len*Math.cos(rad), cannon.y +
cannon.len*Math.sin(rad));
ctx.stroke();
```

- La linea parte dalla base del cannone e punta secondo l'angolo corrente.

c) Disegna proiettili

```
ctx.fillStyle = "red";
bullets.forEach(b => {
  b.x += b.vx;
  b.y += b.vy;
  ctx.beginPath();
  ctx.arc(b.x, b.y, 5, 0, 2*Math.PI);
  ctx.fill();
});
```

- Aggiorna posizione di ogni proiettile.
- Disegna un cerchietto rosso di raggio 5 px.

d) Disegna bolle

```
ctx.fillStyle = "blue";
bolle.forEach(b => {
  if (!b.colpita) {
    ctx.beginPath();
    ctx.arc(b.x, b.y, b.r, 0, 2*Math.PI);
    ctx.fill();
  }
});
```

- Solo le bolle non ancora colpite vengono disegnate.

e) Controllo collisioni

```
bullets.forEach(p => {
  bolle.forEach(b => {
    const dx = p.x - b.x, dy = p.y - b.y;
    if (!b.colpita && Math.sqrt(dx*dx + dy*dy) < b.r + 5) b.colpita = true;
  });
});
```

- Calcola distanza tra proiettile e bolla.
- Se la distanza è minore della somma dei raggi ($b.r + 5$), la bolla viene marcata come colpita.

f) Rigenerazione bolle

```
if (bolle.every(b => b.colpita)) creaBolle();
```

- Se tutte le bolle sono colpite, se ne generano di nuove.

g) Loop continuo

```
requestAnimationFrame(disegna);
```

- Chiama `disegna()` circa 60 volte al secondo per animare il gioco.

7 Funzionamento generale

1. Il cannone è fisso nella parte inferiore del canvas ma può ruotare a sinistra o destra.
2. Premendo “Spara”, viene creato un proiettile con direzione calcolata dall’angolo.

3. Le bolle blu si trovano in alto; se un proiettile le colpisce, spariscono.
4. Quando tutte le bolle sono colpite, ne compaiono di nuove.
5. La matematica (trigonometria) serve a calcolare correttamente traiettoria e velocità dei proiettili.

In pratica, è un **mini gioco didattico** che unisce **grafica 2D, trigonometria e programmazione in JavaScript**.

Codice

```
<!DOCTYPE html>
<html lang="it">
<head>
<meta charset="UTF-8">
<title>Mini Cannone</title>
<style>
body { text-align: center; background: #eef; margin: 0; font-family: sans-serif; }
canvas { background: #ccc; margin: 20px auto; display: block; border: 1px solid #000; }
button { margin: 5px; padding: 10px 15px; font-size: 16px; }
#description {
max-width: 600px;
margin: 20px auto;
padding: 15px;
background: #fff;
border: 1px solid #ccc;
border-radius: 8px;
text-align: left;
}
code {
background: #f0f0f0;
padding: 2px 5px;
border-radius: 4px;
}
</style>
</head>
<body>
<h3>Cannone 2D</h3>
<canvas id="canvas" width="600" height="400"></canvas>
<div>
<button onclick="angle -= 5"> ← SX</button>
<button onclick="angle += 5"> → DX</button>
<button onclick="spara()">💣 Spara</button>
</div>
</body>
</html>
```

</div>

<div id="description">

<h3>Mini Progetto Didattico – IFTS ITIS Grassi</h3>

<p>

Questo semplice gioco 2D è stato realizzato durante un laboratorio pratico presso l'ITIS Grassi, all'interno del percorso IFTS in Informatica e Telecomunicazioni.

</p>

<p>

Il gioco ha come obiettivo colpire le bolle blu con dei proiettili sparati da un cannone ruotabile. È stato programmato in JavaScript usando il tag <code><canvas></code> per la grafica.

</p>

<p>

Per calcolare la traiettoria del proiettile è stata utilizzata una proporzione trigonometrica:

</p>

<p style="text-align: center; font-size: 1.2em;">

<code>x : y = cos(θ) : sin(θ)</code>

</p>

<p>

Questa proporzione ci ha permesso di definire la direzione e la velocità dei colpi in base all'angolo del cannone. Il progetto ha aiutato la classe a comprendere meglio la relazione tra matematica, fisica e programmazione.

</p>

</div>

<script>

const canvas = document.getElementById("canvas");

const ctx = canvas.getContext("2d");

let angle = 0;

const cannon = { x: 300, y: 350, len: 60 };

const bullets = [];

let bolle = [];

function creaBolle() {

bolle = [];

for (let i = 0; i < 10; i++) {

bolle.push({ x: Math.random()*560+20, y: Math.random()*200+20, r: 15, colpita: false });

}

}

function spara() {

```
const rad = angle * Math.PI / 180;
bullets.push({
  x: cannon.x + cannon.len * Math.cos(rad),
  y: cannon.y + cannon.len * Math.sin(rad),
  vx: 5 * Math.cos(rad),
  vy: 5 * Math.sin(rad)
});
}
```

```
function disegna() {
  ctx.clearRect(0, 0, 600, 400);
```

```
  // cannone
  ctx.strokeStyle = "black";
  ctx.lineWidth = 8;
  const rad = angle * Math.PI / 180;
  ctx.beginPath();
  ctx.moveTo(cannon.x, cannon.y);
  ctx.lineTo(cannon.x + cannon.len * Math.cos(rad), cannon.y + cannon.len * Math.sin(rad));
  ctx.stroke();
```

```
  // proiettili
  ctx.fillStyle = "red";
  bullets.forEach(b => {
    b.x += b.vx; b.y += b.vy;
    ctx.beginPath();
    ctx.arc(b.x, b.y, 5, 0, 2 * Math.PI);
    ctx.fill();
  });
```

```
  // bolle
  ctx.fillStyle = "blue";
  bolle.forEach(b => {
    if (!b.colpita) {
      ctx.beginPath();
      ctx.arc(b.x, b.y, b.r, 0, 2 * Math.PI);
      ctx.fill();
    }
  });
```

```
  // collisioni
  bullets.forEach(p => {
    bolle.forEach(b => {
```

```
const dx = p.x - b.x, dy = p.y - b.y;  
if (!b.colpita && Math.sqrt(dx*dx + dy*dy) < b.r + 5) b.colpita = true;  
});  
});
```

```
// rigenera  
if (bolle.every(b => b.colpita)) creaBolle();
```

```
requestAnimationFrame(disegna);  
}
```

```
creaBolle();  
disegna();  
</script>  
</body>  
</html>
```